Test Report — Tourist Safety Monitoring System

Project root: `D:\TouristSafetyMonitoring-src` Date of this run: 2026-10-18 Toolchain: JDK 21 (Temurin, `D:\AndroidTools\jdk-21`), Gradle 8.11.1, Android SDK platform 35, Python 3.11.9 (numpy 2.4.4, pandas 3.0.6, scikit-learn 1.8.0, matplotlib 3.11.2).

> **Honesty statement.** Every result below was produced by an actual command run in this environment and read from the machine-generated XML / JSON / CSV artefacts. No test is marked PASS that was not executed, and nothing is marked as executed on a device — there is no device or emulator here. Where a check could not be run it is listed as **NOT EXECUTED** with the reason.

---

1. How the tests were run (exact commands)

```
# Android JVM unit tests (JUnit4 + Robolectric), from the repo root
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:testDebugUnitTest" "--console=plain"

# Compile-check the instrumented (on-device) sources WITHOUT running them
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:compileDebugAndroidTestKotlin" "--console=plain"

# Build the installable debug APK
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:assembleDebug" "--console=plain"

# Python / ML tests
cd ml
python -m pytest tests -q                    # 86
python -m pytest tests future_risk/tests -q   # 179 (all of them)
```

Raw evidence:

- `app/build/test-results/testDebugUnitTest/*.xml` — per-class JUnit XML.
- `app/build/reports/tests/testDebugUnitTest/index.html` — human-readable HTML report.
- `backend/` — the pytest console summary (`python -m pytest backend/tests -q`) and the `verify_integration.py` pass/fail line per check.
- `ml/.pytest_cache/` and the console summary from pytest.

---

2. Summary

Suite	Command	Executed	Passed	Failed	Errors	Skipped
Android JVM unit (JUnit4 + Robolectric)	`:app:testDebugUnitTest`	**548**	**541**	0	0	7
Authority backend (pytest, in-process)	`python -m pytest backend/tests -q`	**265**	**265**	0	0	0
Authority backend acceptance (real subprocess)	`python backend/verify_integration.py`	**41**	**41**	0	0	0
Python / ML (`pytest`)	`python -m pytest ml/tests ml/future_risk/tests -q`	**179**	**179**	0	0	0
Android instrumented (`androidTest`)	`:app:compileDebugAndroidTestKotlin`	0 (compile only)	—	—	—	—

The **7 skipped** Android cases are the opt-in live-backend tests (`AuthorityLiveBackendTest`). They run only when a server address is passed with `-Dtsm.e2e.baseUrl`, so an ordinary `:app:testDebugUnitTest` run skips them on purpose. They were executed against a real Flask server this round and all seven passed — see §7.1.

Android JVM breakdown (from the JUnit XML, 50 classes — ★ marks what the self-service-registration round touched, ▶ the earlier future-risk round, ◆ the role-separation round, ✚ the LAN-cleartext round):

Test class	Tests
★ `AuthorityApiTest`	28
`PolygonGeometryTest`	22
`TilePolicyTest`	21

Test class	Tests
▶ `FutureRiskAnalyzerTest`	18
▶ `FutureRiskCardModelTest`	18
`PolygonValidatorTest`	18
★ `RegistrationFormTest`	18
`GeoFenceEngineTest`	17
`AuthoritySyncTransportTest`	15
`GpsStatusTest`	15
★ `AuthorityHttpServerTest`	14
`GeoFenceEditorViewModelTest`	14
▶ `FutureRiskFeaturesTest`	13
`PolygonContainmentServiceTest`	13
`TrackingPipelineTest`	13
▶ `FutureRiskReporterTest`	12
`RoomPersistenceTest`	12
`TrackingRecoveryTest`	12
★ `AuthorityViewModelTest`	11
`GeofenceDiagnosticsTest`	11
`MercatorProjectionTest`	11
`PlaceholderWatchTest`	11
`SimulationPathBuilderTest`	11
`ApproachBriefingTest`	10
`ApproachRiskTest`	10
▶ `FutureRiskReportPolicyTest`	10
`MovementAnalyzerTest`	10
◆ `OfficerViewModelTest`	10
`OutboxPayloadsTest`	10
`GeofenceWalkthroughTest`	9
`JsonTest`	9
`PolygonCodecTest`	9
◆ `AuthoritySessionStoreTest`	8
▶ `FutureRiskMovementIntegrationTest`	8
▶ `FutureRiskPayloadTest`	8
`OutboxQueueTest`	8
`SimulationControllerTest`	8
★ `AuthorityLiveBackendTest`	7 (7 skipped by default)
`EmergencyContactRepositoryTest`	7
`MovementControllerTest`	7
✚ `NetworkSecurityConfigTest`	3
▶ `FutureRiskReferenceTest`	6
`PositionReportPolicyTest`	6

Test class	Tests
`PositionReporterTest`	6
`SampleAreasTest`	6
`TileDiskCacheTest`	6
`WatchControllerTest`	6
`ZoneHistoryTest`	6
`SosDeliveryStateTest`	4
`ForestReferenceTest`	3
Total	**548**

Authority backend breakdown (265 tests across ten files, in-process Flask test client + temporary SQLite + a frozen clock):

File	Tests	What it covers
`test_registration.py`	**45**	self-service registration, new this round: a stranger with no credential registers and receives a mints-the-id response with **no token and no secret**; ids are `TR-<year>-<n>` and unique across many registrations, and a colliding text id steps over to the next number; the PIN is scrypt-hashed and neither salt nor digest is ever returned; the `password_*` columns hold an unguessable placeholder that is never the PIN; contacts are validated and stored with the new account; field-level validation; and **no privilege escalation** (`"role": "authority"` is 400, no session is issued, a duplicate phone is 409)
`test_payloads.py`	42	the per-kind payload validators the phone's bytes must satisfy (ranges, whole numbers, `simulated` as a real bool, contact-list shape)
`test_authority.py`	33	the officer routes: overview counts, the SOS queue and acknowledgement, the register, per-tourist detail, one-time-password issuance
`test_sync.py`	29	the sync contract: per-entry verdicts, idempotency by `dedupeKey` , `DUPLICATE` semantics, `/me` delivery totals, latest SOS/position
`test_auth.py`	26	login/logout, token expiry and revocation, role separation, the 400-vs-401 password rules
`test_pin.py`	18	tourist id + PIN sign-in: PIN login, wrong-PIN indistinguishability from a wrong password, `PUT /me/pin` , and that an operator or an account without a PIN can never use one
`test_app.py`	16	app wiring, `/health` `/healthz` , static dashboard serving, error-envelope shape
`test_contacts.py`	15	the emergency-contact replace path and its ordering/validation
`test_security.py`	13	no password (or hash/salt) appears in source, defaults, fixtures or any response
`test_risk.py`	**28**	the `RISK` prediction kind end to end: ingest + idempotency (`risk:<timestampMillis>` dedupe), read-back through `/me.latestRisk` , `/authority/risk` and per-tourist detail, the wire-level `PREDICTION` labelling with its disclaimer, a nullable probability, **12 independent validation rejections** (unreadable band, probability outside [0,1], non-positive and absurd horizon, missing model version, zero window samples, malformed/missing data-quality blocks, >5 factors, a factor without a label, a missing timestamp), and the invariant that **a prediction is not an incident** (a HIGH band leaves `latestSos` empty, `openSosCount == 0` and `state != "SOS"`)

Python / ML breakdown (179 tests across seven files):

File	Tests	What it covers
`ml/tests/test_pipeline.py`	21	GeoLife `.plt` schema, cleaning, feature engineering, grouped split, reproducible training, inference
`ml/tests/test_har_models.py`	21	the shipped activity + impact models: tree schema, count-not-fraction regression, scikit-learn agreement, feature-order contract, model-card honesty, app-asset byte-equality
`ml/tests/test_history.py`	13	historical trajectory helpers: crossing detection against a polygon, hour/weekday/month bucketing, revisit aggregation — on fixtures, no network
`ml/tests/test_osm_eda.py`	12	the Overpass-JSON parser and aggregation (element types, tag keys, feature classes, density) on a hand-written fixture — no network
`ml/tests/test_weather_eda.py`	11	the weather aggregation and the documented harsh-day threshold on a fixture — no network
`ml/tests/test_transport_mode.py`	8	transport-mode segment features and the subject-disjoint split rules — on fixtures
`ml/future_risk/tests/test_future_risk.py`	**93**	the future-event pipeline, see the next table

`ml/future_risk/tests/test_future_risk.py` in detail (93 tests, grouped by test class):

Class	Tests	What it covers
`TestFeatureSchema`	6	the vector is the width the shipped forest indexes into; every feature has a human label and formula; the geometry matches the Kotlin constants; `extract` vs `extract_one` agree and are deterministic
`TestFeatureArithmetic`	10	hand-computed values on constant, free-fall, hard-landing, oscillating, quarter-turn and cancelling signals; correlations; `to_g` ; the short-window and mismatched-channel refusals
`TestLabelGeometry`	8	rebuilt from the raw `full_data.mat` (not the saved summary): the impact is the trial `argmax` ; every positive window ends **before** its own impact and 1..horizon samples before it; negatives come only from ADL trials; `check_no_future_information()` passes on real data; the 10-sample stride; the window matrix is in g
`TestDataQuality`	2	a NaN recording is dropped **and counted**, not repaired; a clean set is returned unchanged
`TestSplits`	3	the split is subject-disjoint, reproducible from the seed alone, and covers every window exactly once
`TestCalibration`	6	Platt scaling is the documented map, monotone and clipped at the same `1e-6` the Kotlin uses; the reliability table's shape; calibration improves Brier on held-out subjects
`TestRiskBands`	5	the band rule is inclusive, ordered and fitted on validation only; the thresholds travel inside the shipped asset
`TestShippedArtefact`	12	the 22-feature contract, class names, geometry and version; the forest is exported in **exactly** the shape `DecisionForest` walks (leaf markers, index bounds, integral counts); feature stats are complete and non-degenerate; the size budget; the app asset is **byte-identical** to the pipeline output
`TestReferenceCases`	4	16 real windows with ≥3 distinct leads; each case's stored features and window are **re-derived** by the extractor; the windows are real recordings, not synthetic
`TestMeasuredResults`	14	the held-out split is disjoint; a majority baseline is reported *and beaten*; the confusion matrix agrees with precision/recall/F1; the **shuffled-label control is at chance**; trial-level sensitivity is reported separately from window recall; per-activity false alarms; the lead-time table; post-impact windows are a diagnostic; the horizon's justification is recorded
`TestModelSelection`	5	the selection rule is re-applied from `tuning.json` (best validation PR-AUC **within** the size budget); the unconstrained winner was rejected **for size, not quality**; the shipped hyper-parameters are the selected ones; the asset is that export plus small metadata
`TestProvenance`	8	the archive hash and size are pinned; the source is `full_data.mat` **, not the pre-cut windows**; the licence position ("research use, no OSI licence"); the corrupt recording recorded; the 6 no-impact fall trials counted; the subject table
`TestOutputsOnDisk`	5	every promised artefact exists; the figures are real non-empty PNGs; the saved window set still describes 51,128 windows; the test predictions cover every held-out window; the reliability CSV matches the metrics table
`TestDocumentation`	5	the model card exists at the path the artefact points to, quotes the **measured** numbers rather than rounded claims, states that the shipped model is **not** the best available, and says what the model cannot do

Plus the eight future-risk classes in the Android suite (93 tests) and `backend/tests/test_risk.py` (28 tests) — the same feature contract, the same shipped asset and the same wire format are therefore checked from Python, from Kotlin and from the server.

---

3. The 148 enumerated test cases

Legend — **Result**: PASS = executed and matched the expectation.

ID	Area	Objective	Expected	Actual	Result	Evidence
GEOF-01	Geofencing	A good fix well inside a polygon is classified `INSIDE`	`INSIDE`	`INSIDE`	PASS	`PolygonContainmentServiceTest › good fix well`
GEOF-02	Geofencing	A good fix well outside is classified `OUTSIDE`	`OUTSIDE`	`OUTSIDE`	PASS	`PolygonContainmentServiceTest › good fix well`
GEOF-03	Geofencing	A fix exactly on the edge is `BOUNDARY` , not a guess	`BOUNDARY`	`BOUNDARY`	PASS	`PolygonContainmentServiceTest › point exactly`
GEOF-04	Geofencing	A fix just inside the tolerance band is `BOUNDARY`	`BOUNDARY`	`BOUNDARY`	PASS	`PolygonContainmentServiceTest › point just in`
GEOF-05	Geofencing	Poor accuracy straddling the edge yields `UNCERTAIN` (never a fabricated inside/outside)	`UNCERTAIN`	`UNCERTAIN`	PASS	`PolygonContainmentServiceTest › poor accuracy not a guess`
GEOF-06	Geofencing	Accuracy worse than the usable maximum yields `UNCERTAIN` everywhere	`UNCERTAIN`	`UNCERTAIN`	PASS	`PolygonContainmentServiceTest › accuracy wors everywhere`

ID	Area	Objective	Expected	Actual	Result	Evidence
GEOF-07	Geofencing	A concave polygon's notch is genuinely outside (true ray-casting, not a radius)	outside	outside	PASS	`PolygonContainmentServiceTest › concave polyg`
GEOF-08	Geofencing	Containment is independent of vertex winding order	same result both orders	same	PASS	`PolygonGeometryTest › containment does not de`
GEOF-09	Geofencing	A polygon with fewer than 3 vertices contains nothing	never inside	never inside	PASS	`PolygonGeometryTest › polygon with fewer than`
GEOF-10	Geofencing	Distance to the boundary is measured in **metres**, not degrees	metres	metres	PASS	`PolygonGeometryTest › distance to boundary is`
GEOF-11	Geofencing	A self-intersecting "bow-tie" polygon is rejected	rejected	rejected	PASS	`PolygonValidatorTest › self-intersecting bow`
GEOF-12	Geofencing	Consecutive duplicate vertices are rejected	rejected	rejected	PASS	`PolygonValidatorTest › consecutive duplicate`
GEOF-13	Geofencing	0/1/2 vertices are rejected and the draft reports how many are still needed	rejected + count	rejected + count	PASS	`PolygonValidatorTest › zero one and two verti` `how many vertices are still needed`
GEOF-14	Geofencing	Two overlapping zones are evaluated independently	independent	independent	PASS	`GeoFenceEngineTest › overlapping zones are ev`
GEOF-15	Geofencing	Hysteresis stops alert flapping while standing on the boundary	no repeat alerts	no repeat alerts	PASS	`GeoFenceEngineTest › hysteresis stops flappin`
GEOF-16	Geofencing	A disabled zone is ignored entirely	ignored	ignored	PASS	`GeoFenceEngineTest › disabled zones are ignor`
GEOF-17	Geofencing	Moving outside→inside a restricted zone raises **exactly one** ENTRY	1 ENTRY	1 ENTRY	PASS	`GeoFenceEngineTest › moving from outside to i` `ENTRY`
GEOF-18	Geofencing	Leaving a restricted zone raises **exactly one** EXIT, recorded but not a warning	1 EXIT, no warning	1 EXIT, no warning	PASS	`GeoFenceEngineTest › leaving a restricted zon`
GPS-01	GPS/tracking	The real pipeline raises one ENTRY and persists it to the database	1 ENTRY + row	1 ENTRY + row	PASS	`TrackingPipelineTest › outside to inside a re` `persists it`
GPS-02	GPS/tracking	Staying inside never repeats the alert	no repeat	no repeat	PASS	`TrackingPipelineTest › staying inside a zone`
GPS-03	GPS/tracking	Overlapping zones each alert independently through the real pipeline	both alert	both alert	PASS	`TrackingPipelineTest › overlapping zones each`
GPS-04	GPS/tracking	A reading delivered via the location subscription reaches DB **and** UI	DB + UI updated	both updated	PASS	`TrackingPipelineTest › a reading delivered th` `and the ui`
GPS-05	GPS/tracking	Tracking refuses to start when location services are off	refuses	refuses	PASS	`TrackingPipelineTest › tracking refuses to st`
GPS-06	GPS/tracking	Starting twice opens exactly one location subscription (no leak)	1 subscription	1 subscription	PASS	`TrackingPipelineTest › starting twice opens e`
OFF-01	Offline queue	A queued event survives the process being killed (real Room/SQLite)	durable	durable	PASS	`OutboxQueueTest › the queue survives the proc`
OFF-02	Offline queue	Enqueue is idempotent on the dedupe key — no duplicates	1 row	1 row	PASS	`OutboxQueueTest › enqueue is idempotent on th`
OFF-03	Offline queue	A transient failure stays pending and increments the attempt count	pending, +1	pending, +1	PASS	`OutboxQueueTest › a transient failure stays p`

ID	Area	Objective	Expected	Actual	Result	Evidence
OFF-04	Offline queue	A delivered event leaves the pending queue and is marked sent	left + sent	left + sent	PASS	`OutboxQueueTest › a delivered event leaves th`
OFF-05	Offline queue	A permanent rejection is marked failed and not retried	failed, no retry	failed, no retry	PASS	`OutboxQueueTest › a permanent rejection is ma`
OFF-06	Offline queue	An event is delivered **exactly once** when the network comes back	exactly once	exactly once	PASS	`OutboxQueueTest › an event is delivered exact`
WATCH-01	Smartwatch	The watch face reflects the **real** tracking fix, not a synthetic coordinate	real fix	real fix	PASS	`WatchControllerTest › the watch reflects the`
WATCH-02	Smartwatch	Entering a restricted zone vibrates **once**, not on every fix	1 vibration	1 vibration	PASS	`WatchControllerTest › entering a restricted z`
WATCH-03	Smartwatch	A genuine re-entry vibrates again	vibrates again	vibrates again	PASS	`WatchControllerTest › a genuine re-entry vibr`
WATCH-04	Smartwatch	SOS queues a durable event, vibrates with the SOS pattern, and stays honestly "queued" offline	queued + SOS buzz	queued + SOS buzz	PASS	`WatchControllerTest › sos queues a durable ev` `offline`
WATCH-05	Smartwatch	SOS is reported **delivered** only when the transport accepts it	delivered	delivered	PASS	`WatchControllerTest › sos is reported deliver`
ML-01	ML	The `.plt` schema is verified from real files (6 header lines, 7 data tokens)	schema matches	schema matches	PASS	`tests/test_pipeline.py::test_plt_schema_verif`
ML-02	ML	The train/test split is grouped by trajectory so no trajectory leaks across splits	0 overlap	0 overlap	PASS	`tests/test_pipeline.py::test_grouped_split_pr`
ML-03	ML	Training is reproducible with the fixed seed	identical model	identical model	PASS	`tests/test_pipeline.py::test_reproducible_tra`
ML-04	ML	Rolling features do not cross trajectory boundaries	bounded per traj	bounded per traj	PASS	`tests/test_pipeline.py::test_features_do_not_`
ML-05	ML	Inference on a valid scored CSV works end to end	runs + finite scores	runs + finite scores	PASS	`tests/test_pipeline.py::test_inference_valid_`
MAP-01	Sample areas	"Add 3 sample areas" produces exactly three zones, one of each type (restricted/dangerous/safe)	3 zones, one per type	3 zones, one per type	PASS	`SampleAreasTest › create makes three areas, o`
MAP-02	Sample areas	Each generated area is a genuinely valid 4-vertex polygon of the requested size	valid, 140 m square	valid, 140 m square	PASS	`SampleAreasTest › create makes three areas, o`
MAP-03	Sample areas	The areas are laid out in a row, one spacing apart, and cannot overlap	~340 m apart, disjoint	~340 m apart, disjoint	PASS	`SampleAreasTest › the areas form a row, one s`
MAP-04	Sample areas	The areas stay near the user, with the middle one centred on the requested point	middle at the user, all within one spacing	as expected	PASS	`SampleAreasTest › the middle area is centred` `close to the user`
MAP-05	Sample areas	Longitude spacing widens towards the poles so the true distance stays ~340 m	wider at 60°N; ~340 m both	as expected	PASS	`SampleAreasTest › longitude spacing widens to`
MAP-06	Sample areas	Every generated area is named/described as a **sample**, never as real data	name starts `Sample:`, description says sample	as expected	PASS	`SampleAreasTest › every area is labelled as a`
MAP-07	Tile servers	A tile template expands to the correct slippy-map URL	`…/{z}/{x}/{y}.png` → real URL	correct URL	PASS	`TilePolicyTest › urlFor expands the slippy-ma`

ID	Area	Objective	Expected	Actual	Result	Evidence
MAP-08	Tile servers	Every mirror is a well-formed template on a **distinct** host (fail-over is real)	≥2 mirrors, all distinct	3 mirrors, distinct	PASS	`TilePolicyTest › every configured host is a u`
MAP-09	Tile servers	The mirror that worked last time is tried first, and the rest still wrap around	preferred first, all present	as expected	PASS	`TilePolicyTest › hostsFrom prefers the rememb`
MAP-10	Tile servers	Out-of-range/negative mirror indices behave sanely (no crash, no lost host)	wraps, all hosts	wraps, all hosts	PASS	`TilePolicyTest › hostsFrom tolerates out-of-r`
MAP-11	Tile cache	A tile written to the on-disk cache is read back byte for byte	identical bytes	identical bytes	PASS	`TileDiskCacheTest › a written tile is read ba`
MAP-12	Tile cache	Reading a tile that was never cached returns null instead of throwing	null, no crash	null, no crash	PASS	`TileDiskCacheTest › reading a tile that was n`
MAP-13	Tile cache	The cache reports a correct tile count and byte size	2 tiles / 150 B	2 tiles / 150 B	PASS	`TileDiskCacheTest › the cache reports its til`
MAP-14	Tile cache	When the cache exceeds its budget it drops the **oldest** tiles first and keeps the newest	≤ budget, newest kept	as expected	PASS	`TileDiskCacheTest › trim drops the oldest til`
MAP-15	Tile cache	Nothing is evicted while the cache is within budget	no evictions	no evictions	PASS	`TilePolicyTest › nothing is evicted while the cache fits · … an input order that is not sort`
MAP-16	Tile cache	Clearing the cache removes every tile	empty	empty	PASS	`TileDiskCacheTest › clear removes everything`
MAP-17	Tile providers	At least **three** independent key-free providers exist, with unique ids (ids are cache folders), and the default is the canonical key-free source	≥3 providers, unique ids	3 providers, unique ids	PASS	`TilePolicyTest › there are at least three ind… is the default provider because it is the cano`
MAP-18	Tile providers	Every provider template is https, well formed, and on a **different hostname** so fail-over is real, and no watermarking provider is offered	all valid, hosts distinct	all valid, hosts distinct	PASS	`TilePolicyTest › every provider template is w… really are on different hostnames so fail-over… all, because it now answers with a watermark i`
MAP-19	Tile providers	URL expansion honours the template's own token order (Esri is `{z}/{y}/{x}`)	`{z}/{y}/{x}` respected	as expected	PASS	`TilePolicyTest › urlFor honours template orde`
MAP-20	Tile providers	Cache entries are **provider-scoped**, so two map styles can never be mixed up	different names per provider	as expected	PASS	`TilePolicyTest › cacheName is provider-scoped`
MAP-21	Tile providers	The fail-over order keeps the remembered provider **first** and silently drops nobody	preferred first, all present	as expected	PASS	`TilePolicyTest › providersToTry keeps the rem`
MAP-22	Tile providers	A provider already known to be unreachable is skipped, and "all down" yields an empty list (give up and say so)	down ones dropped; empty when all down	as expected	PASS	`TilePolicyTest › providersToTry drops the pro… providersToTry is empty when every provider is`
EDIT-01	Geo-fence editor	A polygon corner can be added by **typing** its latitude and longitude, and a comma decimal separator is accepted	vertex added; comma parsed	as expected	PASS	`GeoFenceEditorViewModelTest › a vertex can be… coordinates accept a comma as the decimal sepa`
EDIT-02	Geo-fence editor	An out-of-range latitude or unparseable text is refused **with a reason**, never silently	refused + message	refused + message	PASS	`GeoFenceEditorViewModelTest › an out-of-range… … nonsense typed into the coordinate fields i`

ID	Area	Objective	Expected	Actual	Result	Evidence
EDIT-03	Geo-fence editor	A complete polygon can be drawn purely from typed coordinates and saved	3 vertices, saved	3 vertices, saved	PASS	`GeoFenceEditorViewModelTest › a whole polygon`
EDIT-04	Geo-fence editor	A tap that lands on the previous corner is refused **and explains itself** (the "I have to tap twice" report)	refused + message, count unchanged	as expected	PASS	`GeoFenceEditorViewModelTest › a tap on the pr… a duplicate` · `… a refused duplicate tap explai`
MOV-01	Movement model	The **shipped** activity forest reproduces scikit-learn's own probabilities on real recorded windows, on the JVM	same class + probability	same	PASS	`ForestReferenceTest › the shipped activity mo… windows`
MOV-02	Movement model	The **shipped** impact/fall forest does the same	same class + probability	same	PASS	`ForestReferenceTest › the shipped fall model … windows`
MOV-03	Movement model	Both shipped models index the same 24 features in the same order as the extractor — the Python↔Kotlin contract	identical order	identical order	PASS	`ForestReferenceTest › both shipped models ind… extractor`
MOV-04	Movement model	Impact **+** the trained signature **+** stillness is reported as a *suspected* fall	suspected fall	suspected fall	PASS	`MovementAnalyzerTest › impact plus the traine… fall`
MOV-05	Movement model	A phone lying still is **never** a fall — there is no impact, so the physics gate never opens	not a fall	not a fall	PASS	`MovementAnalyzerTest › a phone lying still is`
MOV-06	Movement model	A large impact the model does **not** recognise is not reported as a fall	not a fall	not a fall	PASS	`MovementAnalyzerTest › an impact the model do`
MOV-07	Movement model	An impact followed by continued movement fails the stillness gate	not a fall	not a fall	PASS	`MovementAnalyzerTest › an impact followed by`
MOV-08	Movement model	One fall seen by five overlapping windows raises **exactly one** alert	1 alert	1 alert	PASS	`MovementAnalyzerTest › one fall seen by five`
MOV-09	Movement model	A second, later fall is alerted again once the cooldown has elapsed	alerted again	alerted again	PASS	`MovementAnalyzerTest › a second fall after th`
MOV-10	Movement model	An impact landing in the last quarter of a window is **not** failed for a tail it cannot measure (the gate is skipped)	skipped, not failed	skipped, not failed	PASS	`MovementAnalyzerTest › a window whose impact … unmeasurable tail`
MOV-11	Movement model	Without the trained model the state says so instead of guessing	"unavailable"	"unavailable"	PASS	`MovementAnalyzerTest › without a fall model t`
MOV-12	Movement model	Activity is reported only when an activity model is actually present	no activity claimed	no activity claimed	PASS	`MovementAnalyzerTest › activity is reported o`
MOV-13	Movement model	Implausible sensor samples are detected rather than averaged into the window	detected	detected	PASS	`MovementAnalyzerTest › implausible sensor sam`
MOV-14	Movement model	The watch opens exactly **one** sensor subscription, survives a second start, releases on stop, and reports a device lacking the sensors instead of pretending	1 subscription; reported	as expected	PASS	`MovementControllerTest › starting twice does … device without both sensors is reported, not p… and can be repeated`
MOV-15	Movement model	A build without the trained model says so instead of running blind; an implausible driver reading is skipped and reported	reported	reported	PASS	`MovementControllerTest › a build without the … blind` · `… an implausible driver reading is ski`

ID	Area	Objective	Expected	Actual	Result	Evidence
MOV-16	Movement model	Exported tree `value` arrays are real **integer class counts**, not rounded fractions — regression guard for the exporter bug that would have shipped a wrong model	counts; root sum ≥ 100	as expected	PASS	`ml/tests/test_har_models.py::test_exported_va`
MOV-17	Movement model	The app ships **byte-identical** copies of the verified training artefacts, so "retrained but forgot to copy" cannot pass quietly	identical bytes	identical bytes	PASS	`ml/tests/test_har_models.py::test_app_ships_t`
MOV-18	Movement model	The model card's spec reports the **real** UCI HAR test-split size (2947, not the 2949 it used to claim)	2947 windows	2947 windows	PASS	`ml/tests/test_har_models.py::test_spec_report`
RISK-01	Approach risk	Zone severity is the dominant term and is ordered DANGEROUS > RESTRICTED > SAFE	ordered	ordered	PASS	`ApproachRiskTest › zone severity is the domin`
RISK-02	Approach risk	The tourist's **own** recorded crossings add points, capped so history can never dominate the score	capped	capped	PASS	`ApproachRiskTest › recent crossings count tow`
RISK-03	Approach risk	A zone the tourist has never crossed is flagged as **unknown ground**, not as safe	unknown ground	unknown ground	PASS	`ApproachRiskTest › a zone the tourist has nev`
RISK-04	Approach risk	Proximity scales from nothing at the 200 m radius to full at the boundary, and being inside is full proximity	scales; full inside	as expected	PASS	`ApproachRiskTest › proximity scales from noth being inside the zone counts as full proximity`
RISK-05	Approach risk	Darkness adds points and midday does not	night > day	night > day	PASS	`ApproachRiskTest › darkness adds points, midd`
RISK-06	Approach risk	An **unavailable** movement model is treated as neutral, never as safe	neutral, not safe	neutral, not safe	PASS	`ApproachRiskTest › an unavailable movement mo`
RISK-07	Approach risk	The score never exceeds 100, and the listed factors account for **every** point of it	≤ 100; factors sum to the score	as expected	PASS	`ApproachRiskTest › the score never exceeds 10`
RISK-08	Approach risk	The bands change at 30 and 60, and walking towards the boundary adds a little while standing still does not	LOW/MODERATE/HIGH at 30/60	as expected	PASS	`ApproachRiskTest › bands change at 30 and 60 little, standing still does not`
RISK-09	Approach risk	History is derived **only** from the requested zone, with the 7/30-day windows measured against the supplied clock and future timestamps never counted	per-zone, clock-relative	as expected	PASS	`ZoneHistoryTest › only the requested zone is windows are measured against the supplied cloc a past window`
RISK-10	Approach briefing	A restricted/dangerous zone within 200 m is briefed; a **safe** zone is never briefed however close it is; no position means no briefing	briefed / never / null	as expected	PASS	`ApproachBriefingTest › standing just outside credit · … a safe zone is never briefed, howev briefing`
RISK-11	Approach briefing	When two boundaries are near, the **higher-risk** one is briefed, and equal-risk ties resolve deterministically to the closer boundary so the card cannot flicker between fixes	higher risk; deterministic	as expected	PASS	`ApproachBriefingTest › when two boundaries ar equal-risk zones resolve deterministically to`

ID	Area	Objective	Expected	Actual	Result	Evidence
RISK-12	Approach briefing	A zone far beyond the radius is not briefed; a disabled zone is ignored; being inside is reported as zero distance; the live movement state feeds the score	as expected	as expected	PASS	`ApproachBriefingTest › a zone far beyond the ignored · … being inside the zone is reported feeds the score · … darkness raises the score`
WATCH-06	Smartwatch	A settled SOS delivery outcome is never walked back by a late "still queued" observation, while a newly raised SOS resets the indicator	no regression; new SOS resets	as expected	PASS	`SosDeliveryStateTest` (4 tests)
AUTH-01	Authority client	A base URL without a scheme is accepted as `http` and trailing slashes are dropped	accepted, normalised	as expected	PASS	`AuthorityApiTest › accepts a bare host by ass drops trailing slashes`
AUTH-02	Authority client	An empty base URL resolves to nothing, so the client refuses to send rather than guessing	resolves empty	as expected	PASS	`AuthorityApiTest › an empty base url resolves`
AUTH-03	Authority client	Login carries the role and the device label, and a 200 with no token is a **protocol error**, not an empty success	protocol error	protocol error	PASS	`AuthorityApiTest › parses a login response · label · … a login response without a token is`
AUTH-04	Authority client	The sync body embeds each payload as a nested object (not a quoted string) and keeps a corrupt payload as text so the server can reject it with a reason	object payloads	as expected	PASS	`AuthorityApiTest › sync body embeds the paylo body keeps a corrupt payload as text so the se body carries every event and its kind`
AUTH-05	Authority client	Every delivery verdict the server can return is read, and an unknown status becomes `UNKNOWN` rather than a silent delivery	no silent success	no silent success	PASS	`AuthorityApiTest › reads every delivery verdi status is UNKNOWN rather than silently deliver`
AUTH-06	Authority client	The server's own error code and message are surfaced verbatim, and a profile with no tourist object is a protocol error	verbatim	verbatim	PASS	`AuthorityApiTest › extracts the server's own tourist object is a protocol error`
AUTH-07	Authority transport	A `DELIVERED` verdict is reported with the acknowledgement id; a `DUPLICATE` counts as delivered because the server already holds it	delivered / delivered	as expected	PASS	`AuthoritySyncTransportTest › a delivered verd … a duplicate verdict counts as delivered bec`
AUTH-08	Authority transport	`REJECTED` fails the event with the server's reason; an HTTP 400 fails it because the same bytes can never be accepted	failed + reason	as expected	PASS	`AuthoritySyncTransportTest › a rejected verdi … http 400 fails the event because the same b`
AUTH-09	Authority transport	A 401, an expired token, a 500 and a dead network all **keep the event queued** (a configuration mistake must never discard it)	queued	queued	PASS	`AuthoritySyncTransportTest › http 401 keeps t deliver it · … http 500 keeps the event queued … an expired token is treated as not signed i`
AUTH-10	Authority transport	With no address, or an address but no sign-in, the event stays queued and says why; the selector uses the honest no-backend transport until an address exists	queued + reason	as expected	PASS	`AuthoritySyncTransportTest › with no address · … with an address but no sign-in the event s no-backend transport until an address exists`
AUTH-11	Authority transport	A non-JSON success body, a response with no verdict for our event, or an unrecognised verdict is never treated as a delivery	not delivered	not delivered	PASS	`AuthoritySyncTransportTest › a non-json succe … a response with no verdict for our event ke keeps the event queued`

ID	Area	Objective	Expected	Actual	Result	Evidence
AUTH-12	Real socket	The client delivers over a **real TCP socket** to a hand-rolled HTTP/1.1 server; method, path, bearer header and body are verified byte-for-byte	real HTTP	real HTTP	PASS	`AuthorityHttpServerTest › the transport deliv really reaches the server with the expected by the token back`
AUTH-13	Real socket	A closed port is `Unreachable` (no exception escapes), a gateway error page is an `HttpError`, a truncated body is a protocol error, and a 401 keeps the event queued	no escaped exception	as expected	PASS	`AuthorityHttpServerTest › a closed port is Un … a gateway error page is an HttpError, never protocol error, not a delivery · … a 401 from`
AUTH-14	JSON codec	The hand-rolled codec round-trips nested objects/arrays/unicode, writes whole numbers **without a decimal point** (so the server's integer validator accepts them), rejects truncated/trailing documents, and degrades instead of throwing	round-trip	as expected	PASS	`JsonTest` (9 tests)
AUTH-15	Emergency contacts	`replaceAll` assigns priority from list position, never leaves a partial list, drops what is gone, stores a null relationship, escapes quotes, and uploads the stored order	whole-list replace	as expected	PASS	`EmergencyContactRepositoryTest` (7 tests)
AUTH-16	Outbox payloads	SOS marks itself SIMULATED and tolerates a missing fix; the contacts payload is a whole-list replace with an ordered priority; dedupe keys are stable	as expected	as expected	PASS	`OutboxPayloadsTest` (10 tests)
AUTH-17	Authority client	A PIN sign-in carries the PIN and **omits** the password (an empty password is a 400, so it must not be sent), the set-PIN body is a single field, and a profile's stored `latestPosition` is parsed with its accuracy and simulated flag	PIN only	as expected	PASS	`AuthorityApiTest › pin login body carries the single field · … parses the authority's stored`
POS-01	Offline position sync	A **real** GPS fix is queued as a durable `POSITION` event; a simulated fix and an out-of-range coordinate are **never** queued; nothing is queued until an authority address is configured, and once it is, the pending fix is reported immediately rather than lost	real-only, gated	as expected	PASS	`PositionReporterTest` (6 tests) · `PositionReporte`
POS-02	Position-report policy	The first fix is always reported; a stationary tourist is re-reported once the interval elapses; a ~11 m nudge inside the interval is suppressed but a ~222 m move is reported; a disabled policy and a backwards clock are handled	interval + distance	as expected	PASS	`PositionReportPolicyTest` (6 tests)

ID	Area	Objective	Expected	Actual	Result	Evidence
POS-03	Position payload	The `POSITION` payload carries the coordinates, source and device timestamp, writes `null` accuracy rather than invalid JSON for a non-finite value, and its dedupe key is a function of the fix timestamp	valid JSON	as expected	PASS	`OutboxPayloadsTest › position payload carries` `position payload writes null accuracy rather t` `position dedupe key is a function of the fix t`
FRISK-01	Future-risk features	The 22 features are computed and **frame the forest contract**: hand-checkable values on constant, free-fall, hard-landing, oscillation, quarter-turn and cancelling signals; the "no net direction" guard returns 0 instead of dividing by ~zero; a short window and mismatched channels are refused; the Python and Kotlin feature orders agree	22, hand-checked	as expected	PASS	`FutureRiskFeaturesTest` (13) · `ml/future_risk/t` `TestFeatureSchema` / `TestFeatureArithmetic`
FRISK-02	Model loading + inference consistency	The **shipped** asset, re-walked on the JVM, reproduces scikit-learn's own probabilities for **16 real held-out windows** (feature parity; forest parity worst 5.9e-04, mean < 5e-04 on a step function; Platt calibration); an artefact whose feature **names** do not match is refused rather than mis-indexed; the app asset is **byte-identical** to the pipeline output	parity	as expected	PASS	`FutureRiskReferenceTest` (6) · `ml/future_risk/t` `TestShippedArtefact` / `TestReferenceCases`
FRISK-03	Leakage prevention	Rebuilt from the raw `full_data.mat` : every positive window ends strictly **before** its own impact; negatives come only from ADL trials; the impact is the trial `argmax` ; **no fitted preprocessing exists** at all; a **shuffled-label control** (same features, same pipeline, permuted labels) scores near chance	no leakage	chance control 0.477	PASS	`ml/future_risk/tests` `TestLabelGeometry` / `Test` `ml/outputs/future_risk/metrics.json` (`leakage_`
FRISK-04	Probability ↔ category mapping	The band rule is inclusive and ordered and fitted on **validation only**, and the thresholds travel inside the artefact and the Kotlin constants: **HIGH ≥ 0.314046**, **MODERATE ≥ 0.069201**; the figure displayed is the **calibrated** probability, never the raw vote; LOW is never shown for "we do not know"	ordered, calibrated	as expected	PASS	`FutureRiskAnalyzerTest` (18) · `FutureRiskCardMo` `TestRiskBands`
FRISK-05	Insufficient / implausible / stale data	Fewer than 50 samples reports **insufficient data**; an implausible (> 16 g) or non-finite reading is refused; a forecast older than 10 s is reported **stale**; a paused watch and a build with no model report "prediction unavailable" rather than a guess	honest states	as expected	PASS	`FutureRiskAnalyzerTest` (18) · `FutureRiskMoveme`

ID	Area	Objective	Expected	Actual	Result	Evidence
FRISK-06	Offline queueing + recovery	Against a **real SQLite outbox**: one durable `RISK` row carrying a **summary** (not a sensor stream), nothing queued when no address is configured, queue-then-recover after an offline spell, and stable dedupe; no coordinates or raw samples appear in the payload	durable, summary-only	as expected	PASS	`FutureRiskReporterTest` (12) · `FutureRiskPayloa`
FRISK-07	Distinction & no automatic SOS	The movement watch feeds **both** models from **one** subscription; a HIGH forecast raises **no** fall alert and changes no operational state; the card names the configured-geofence status and the approach-risk checklist as different things and states the factors are **non-causal** and that the model **never sends an SOS**	three things apart	as expected	PASS	`FutureRiskMovementIntegrationTest` (8) · `Future`
FRISK-08	API auth + validation	The `RISK` kind is authenticated like any tourist event, validated at the wire level (**12 independent rejections**: band, probability range, horizon, model version, window samples, data-quality block, factor ceiling, missing fields), `UNAVAILABLE` is rejected outright, ingestion is idempotent by `risk:<timestampMillis>`, and a HIGH prediction leaves `latestSos` empty and `state != "SOS"` — **a prediction is not an incident**	reject / not an incident	as expected	PASS	`backend/tests/test_risk.py` (28)
FRISK-09	Dashboard integration	`GET /authority/risk`, the per-tourist history and the overview's `riskAssessments` / `riskElevated` counts return the fields the new **"Future-risk predictions"** panel renders, labelled `kind: "PREDICTION"` with a disclaimer and kept out of the SOS queue and the incident feed	prediction rows	as expected	PASS	`backend/tests/test_risk.py` (28)
FRISK-10	E2E prediction delivery (no device)	Over a **real HTTP subprocess server**, an outbox batch shaped exactly as the phone builds it carries a `RISK` summary (origin `ON_DEVICE_MODEL`): it is accepted, labelled `PREDICTION`, read back from `/authority/risk` and the tourist detail with its probability and factors, counted as elevated risk while `sosTotal` stays 1, and **survives a server restart**	end-to-end over HTTP	as expected	PASS	`verify_integration.py` (41/41; "the on-device fu dashboard" · "… raises NO incident or SOS" · "the f
ROLE-01	Role separation (server)	The officer register `GET /authority/tourists` is a **server-side** boundary, not a hidden screen: an officer token reads it; a **tourist** token presented to the same route is refused with **403**; an unauthenticated request is **401**	403 for a tourist	as expected	PASS	`backend/tests/test_auth.py` (26) · `AuthorityLiv` register and a tourist token is refused it (a t `AuthorityHttpServerTest › a tourist token ref` not an empty register

ID	Area	Objective	Expected	Actual	Result	Evidence
ROLE-02	Officer credential check	Officer sign-in posts the officer id and password and stores an **officer** session; a refused officer sign-in stores **nothing** and says only that the pair was not accepted (so officer ids cannot be enumerated); an unreachable authority is named as unreachable, not as bad credentials; blank credentials and a blank address are refused **before** any request leaves the phone; a 403 reading the register drops the session	officer session only	as expected	PASS	`OfficerViewModelTest` (10) · `AuthorityLiveBacke` `register and a tourist token is refused it (of`
ROLE-03	Two independent sessions	The tourist session and the officer session are stored under **separate keys**: an officer sign-in does **not** make the phone a tracked tourist; signing the tourist out does **not** evict the officer (and the reverse); both are restored after the store is torn down and rebuilt, together with the configured address	independent, durable	as expected	PASS	`AuthoritySessionStoreTest` (8, real on-disk DataS `sign-in does not sign a tourist in on the same` `clears only the officer session`
ROLE-04	Address no longer gates the tourist	The submit controls are gated on **form validity**, never on a backend address being present: with nothing configured the form still validates field by field, names every missing field, and refuses only what is genuinely invalid	form, not address	as expected	PASS	`RegistrationFormTest` (18: `a complete form repo` `reported against its own field, not as one lum` `the confirmation field while the pin stays cle` `only` , `the outstanding list names only what is` `a blank address is refused before any call is` named reason — the *button* is live)
ROLE-05	Build-time default address	A debug build can carry a default address (`-PAUTHORITY_URL` / `TSM_AUTHORITY_URL` / `local.properties`); it is written to the session store **only when nothing is configured**, and can never overwrite an address the user has chosen	seed once	as expected	PASS	`AuthoritySessionStoreTest › seedBaseUrlIfUnse` `configured · … ignores an empty default so an` release's `BuildConfig.DEFAULT_AUTHORITY_URL ==` packaged dex
ROLE-06	Officer register rendering (client)	The register rows parse with **registration**, **tracking** and the safety signals as three separate claims; a row with no position carries `hasPosition=false` and a null position rather than a fabricated `0,0` ; a row with no Tourist ID is dropped rather than rendered anonymously	labelled, not invented	as expected	PASS	`AuthorityApiTest` +6 (`parseTourists`) · `Authori` `fetched with the officer bearer token and pars` (every live row asserted to carry an id and both sta one)
NET-01	LAN cleartext policy	The app declares a network-security profile that permits the **cleartext** HTTP the LAN authority needs, and the manifest points at it: with `targetSdk 35` and no such profile, **every** call dies with `Cleartext HTTP traffic to <host> not permitted` before a single byte leaves the phone — which the sign-in screens can only report as "the authority is not answering"	cleartext permitted; manifest wired	as expected	PASS	`NetworkSecurityConfigTest` (3 tests) — reads the `res/xml/network_security_config.xml` and fails i `dump xmltree` : `android:networkSecurityConfig=@` `res/xml/network_security_config.xml`)

ID	Area	Objective	Expected	Actual	Result	Evidence
REG-01	Self-registration	A tourist with **no credential at all** registers themselves and receives a response carrying the minted Tourist ID and the account's own non-secret fields — **no token, no password, no PIN, no salt, no digest** — and registering does **not** sign them in	201 + id, no secret	as expected	PASS	`backend/tests/test_registration.py` (45) · `veri` the database mints a TR-- id" · "the registration res · `RegistrationFormTest` (18) · `AuthorityApiTest`
REG-02	Unique Tourist ID	The id is minted by the **database** (`tourist_id_sequence` + the `tourists.id` primary key), not by the client; twelve sequential registrations get twelve distinct ids in `TR-<year>-<5 digits>` form; a colliding text id steps over to the next number; the counter is never reset	unique, DB-minted	as expected	PASS	`backend/tests/test_registration.py` · live: the p
REG-03	Registration validation + uniqueness	Blank/over-long name, blank phone, a PIN that is not 4–8 digits, a mismatched confirmation, a half-filled contact row and too many contacts are all refused with a specific message; a duplicate phone number is **409**, and validation happens **before** any request leaves the phone	400 / 409, specific	as expected	PASS	`backend/tests/test_registration.py` · `Registra` (11) · `AuthorityHttpServerTest` (14)
REG-04	No privilege escalation	The public registration endpoint creates only tourists: it reads **no** privilege field, `"role": "authority"` is `400`, an operator account can never be minted this way, registration issues no session, and the option to self-assign an authority role does not exist	tourist only	as expected	PASS	`backend/tests/test_registration.py` · `verify_i` authority account")
REG-05	Registered-tourist listing	The officer's authenticated `GET /authority/tourists` shows the new account (id, name, **registration date**, source `SELF`) with **registration**, **tracking** and the safety signals in separate fields — a tourist who has not reported yet reads `AWAITING_FIRST_REPORT` / `hasPosition=false` / `latestPosition=null` rather than a fabricated position — and a tourist token is still `403` on that route	authenticated, separated	as expected	PASS	`backend/tests/test_registration.py` · `verify_i` appears on the officer's dashboard..." · "registratio registered account survived the restart") · `Authori` appears on the authority dashboard and then repo officer register is fetched with the officer bearer to
PIN-01	Backend PIN	A tourist can sign in with an id + PIN; a wrong PIN is indistinguishable from a wrong password (same 401 body); `PUT /me/pin` accepts only 4–8 digits; an operator or a PIN-less account can never use a PIN	id + PIN	as expected	PASS	`backend/tests/test_pin.py` (18 tests)

ID	Area	Objective	Expected	Actual	Result	Evidence
ACK-01	Authority acknowledgement	An officer acknowledgement moves SOS `OPEN → ACKNOWLEDGED` with the officer id and time, and `GET /me` returns it to the tourist together with the stored position	ack + position	as expected	PASS	`backend/tests/test_authority.py` · `backend/tes` `AuthorityLiveBackendTest` (`latestSos=OPEN` , `la`
LIVE-01	Live end-to-end	The production Kotlin client against a real `run.py` : **self-registration → the server-minted Tourist ID → signing in with that id and PIN → appearing under *Registered Tourists* with no position → gaining a real position**, SOS delivery + idempotency + officer queue, a position report landing on the authority map, PIN set + PIN sign-in (wrong PIN → 401), the **in-app officer console reading the register with an officer token while a tourist token is refused 403**, a dead backend leaving the event queued with its reason, and an **on-device future-risk summary** stored as a `PREDICTION` that raises no SOS	7/7	as expected	PASS	`AuthorityLiveBackendTest` (7 tests, `skipped="0"`
BE-01	Backend	The sync response is `{results: […], serverTimeMillis}` with one verdict per entry, in order; re-delivering the same batch returns `DUPLICATE` and never duplicates the row	idempotent	idempotent	PASS	`backend/tests/test_sync.py` (29) · `verify_integ` reported DUPLICATE, never duplicated")
BE-02	Backend	SOS position is optional (a tourist with no fix can still call for help); a `simulated` SOS is stored and surfaced as **simulated**, never as real	optional; badged	as expected	PASS	`backend/tests/test_payloads.py` (42) · `verify_i` so it cannot be mistaken for a real one")
BE-03	Backend	A tourist token cannot reach the authority API, authority routes reject an unauthenticated request, and tokens expire and can be revoked	401 / 403	as expected	PASS	`backend/tests/test_auth.py` (26) · `verify_integ` authority API")
BE-04	Backend	No password is hard-coded: the first start generates one, prints it **once**, writes it to a file, and no salt/hash ever leaves the server	generated once	as expected	PASS	`backend/tests/test_security.py` (13) · `verify_i` account (not a hard-coded password)")
BE-05	Backend	An emergency-contact upload replaces the whole list and reads back with a count	replaced	replaced	PASS	`backend/tests/test_contacts.py` (15) · `verify_i` read back")
BE-06	Backend	The officer overview / SOS / events / tourists routes return the counts the dashboard renders, and the SOS workflow moves OPEN → ACKNOWLEDGED	counts + ack	as expected	PASS	`backend/tests/test_authority.py` (33) · `verify_` open queue")

ID	Area	Objective	Expected	Actual	Result	Evidence
BE-07	Backend	Restarting the server leaves the data intact — the SOS, its acknowledgement, the event log **and the future-risk prediction** all survive — and the dashboard page is still served	persisted	persisted	PASS	`verify_integration.py` (33 checks: outbox delive prediction read-back, restart persistence, dashboa

Total enumerated cases: 149 executed (all PASS) + 25 declared NOT EXECUTED (INST-01…INST-25).

---

4. Supplementary executed checks

These also ran and passed in the same suites (they are part of the **541 / 179** Android+ML totals, and cover the key-less map, the sample-area generator, the offline tile cache, the typed-coordinate editor, the movement/fall pipeline, the approach-risk maths, the future-event risk model, the offline position queue, self-service registration, the two-role session split and the authority client):

Area	Coverage	Evidence
Key-less map projection	known slippy-map tile indices, screen↔world round-trips, ground resolution at the equator, latitude clamping, panning directions	`MercatorProjectionTest` (11 tests)
Polygon encoding	ordered/lossless/locale-independent round-trip, malformed tokens skipped, precision preserved	`PolygonCodecTest` (9 tests)
Persistence	durability across close/reopen, pruning, simulated-vs-real isolation, migration contract, and **executing both registered migrations** on a real SQLite database to prove a pre-upgrade row survives	`RoomPersistenceTest` (12 tests)
Editing rules	undo/clear, 3-vertex minimum, duplicate-tap ignored, save validation	`GeoFenceEditorViewModelTest` (14 tests)
Simulator	route builder + controller (real engine, stamped rows)	`SimulationPathBuilderTest` (11), `SimulationControllerTest` (8)
Outbox payloads	dedupe-key stability, quote escaping, simulated SOS marking, missing-fix tolerance, contacts-as-whole-list-replace, and the `POSITION` payload incl. `null` accuracy for a non-finite value	`OutboxPayloadsTest` (10 tests)
Offline position sync	real-GPS-only queueing, simulated/out-of-range rejection, the "no address = no queue" gate, dedupe by fix timestamp, and the interval+distance throttle	`PositionReporterTest` (6 tests), `PositionReportPolicyTest` (6 tests)
Self-registration form	the local rules a tourist meets: blank/long name, blank phone, non-digit PIN, 4–8 digit bounds, mismatched confirmation, contact trimming/re-numbering/null relationship, half-filled contact rows, the ≤20 contact ceiling, that the confirmation is never carried in the draft that is sent, and — new this round — **which field each problem belongs to** and **exactly what is still outstanding** (so a submit control is never silently dead)	`RegistrationFormTest` (18 tests)
Registration request/response	the request body asks for a tourist and carries the PIN and contacts but **never** a password, a confirmation or a privilege field; an empty contact list is still sent; the minted id parses; a response with no `touristId` is a protocol error rather than an empty success; a registration over a **real socket** goes out with **no** Authorization header and a 409 duplicate surfaces the server's own message; and (new) the officer register is fetched with the officer's bearer token, parsed end to end, and a tourist token on the same route comes back a 403	`AuthorityApiTest` (28 tests), `AuthorityHttpServerTest` (14 tests)
Registration state machine	a success stores the minted id for the success screen and signs nobody in; a bad form is refused **before** any request leaves the phone; a 409 reads as "already registered"; an unreachable authority names the address; a blank address is refused; clearing the success screen works; and a later failure does not erase an id the tourist may still be copying	`AuthorityViewModelTest` (11 tests, Robolectric + in-memory Room)
Officer console state machine	officer sign-in stores an officer session and reads the register; it does **not** sign a tourist in; a rejected sign-in stores nothing and says only that the pair was refused; an unreachable authority is named; blank credentials / a blank address are refused before any call; the register request carries the officer bearer token; a 403 drops the session; sign-out revokes the token and clears **only** the officer session; an unsigned-in refresh asks for a sign-in rather than calling the server	`OfficerViewModelTest` (10 tests, Robolectric)

Area	Coverage	Evidence
Session storage on disk	a real `PreferenceDataStore` over a throw-away file, no Robolectric: the address and both sessions survive the store being torn down and rebuilt; a build-time default never overwrites a chosen address and an empty default changes nothing; the tourist and officer sessions are independent, and neither sign-out evicts the other	`AuthoritySessionStoreTest` (8 tests)
Sample-area generator	three valid non-overlapping areas, one per type, near the user; polar longitude scaling	`SampleAreasTest` (6 tests)
Tile servers & cache policy	URL building (incl. `{z}/{y}/{x}` order), provider fail-over order, index wrapping, provider-scoped cache keys, LRU eviction (under budget, over budget, unsorted input, oversized entry, empty cache), the OpenStreetMap concurrent-request cap, and the explicit refusal to offer CARTO	`TilePolicyTest` (21 tests)
Offline tile cache (real files)	byte-exact round-trip, missing tile is null, count/size reporting, forward-slash names, LRU trim on real files, clear	`TileDiskCacheTest` (6 tests)
Android cleartext policy	the manifest declares `android:networkSecurityConfig` pointing at a profile whose `base-config` sets `cleartextTrafficPermitted="true"` , and the manifest never re-blocks it with `usesCleartextTraffic="false"` — the setting that decides whether a plain-HTTP LAN authority is reachable from a real phone	`NetworkSecurityConfigTest` (3 tests)
Movement / fall pipeline	window management, the three fall gates, model-absent handling, one-alert-per-fall cooldown, sensor-subscription ownership, session reset	`MovementAnalyzerTest` (10), `MovementControllerTest` (7)
Exported-forest fidelity	the **shipped Kotlin assets** reproduce scikit-learn's probabilities and argmax on real windows, and both models agree with the extractor on feature order	`ForestReferenceTest` (3 tests)
Approach risk	published weights, history windows, proximity/night/ambulation terms, cap at 100, band boundaries, factor accounting	`ApproachRiskTest` (10), `ZoneHistoryTest` (6), `ApproachBriefingTest` (10)
Watch delivery indicator	the sticky-terminal-state ordering rule	`SosDeliveryStateTest` (4 tests)
ML plumbing	malformed/duplicate/invalid-coordinate removal, feature finiteness, model save/load, invalid-input handling	`ml/tests/test_pipeline.py` (21 tests)
Shipped-model contract	tree schema, integer-count regression, scikit-learn agreement, feature-order contract, spec honesty, app-asset byte-equality	`ml/tests/test_har_models.py` (21 tests)
OSM EDA parser	element-type counts, tag-key ranking, feature-class tallies, density maths — on a hand-written Overpass fixture, no network	`ml/tests/test_osm_eda.py` (12 tests)
Weather EDA maths	aggregation and the documented harsh-day threshold — on a fixture, no network	`ml/tests/test_weather_eda.py` (11 tests)
Trajectory history	crossing detection against a polygon, hour/weekday/month bucketing, revisit aggregation — on fixtures	`ml/tests/test_history.py` (13 tests)
Transport-mode model	segment features and the subject-disjoint split rules — on fixtures	`ml/tests/test_transport_mode.py` (8 tests)
Future-risk features	the 22 features index by index (constant/free-fall/hard-landing/oscillation/rotation/cancelling signals), the "no net direction" guard, the short-window and mismatched-channel refusals, and the `NAMES` order the forest contract depends on	`FutureRiskFeaturesTest` (13 tests)
Shipped future-risk asset	the **shipped** JSON against scikit-learn's own probabilities on **16 real held-out windows**: feature parity, forest parity (worst 5.9e-04, mean < 5e-04 on a step function), Platt calibration parity, and refusal of an artefact whose feature *names* do not match	`FutureRiskReferenceTest` (6 tests)
Future-risk inference	band mapping at each threshold, calibrated-not-raw probability, insufficient data, implausible (>16 g) and non-finite readings, staleness, paused, factor ordering, non-causal wording, determinism	`FutureRiskAnalyzerTest` (18 tests)
Future-risk sharing policy	first-elevated, escalation (never throttled), the 120 s refresh, one "cleared" record, never UNAVAILABLE/stale, clock-jump robustness	`FutureRiskReportPolicyTest` (10 tests)
Future-risk outbox (real SQLite)	one durable `RISK` row, the summary-not-stream payload, no row when unconfigured, offline queue + recovery, dedupe	`FutureRiskReporterTest` (12 tests)
Future-risk wire contract	through a real JSON parser: null probability, non-finite handling, escaping, and **no** coordinates or raw samples in the payload	`FutureRiskPayloadTest` (8 tests)

Area	Coverage	Evidence
One sensor feed, two models	the movement watch feeds the risk model from the same subscription; no model → "prediction unavailable"; a HIGH forecast raises **no** fall alert; implausible windows refused by both; stop keeps the last forecast	`FutureRiskMovementIntegrationTest` (8 tests)
What the card may say	no band when there is no prediction, **never** "low" for "we do not know", thresholds printed with their provenance, the non-causal caveat, and that the model **never** claims to summon help	`FutureRiskCardModelTest` (18 tests)
Future-risk pipeline (Python)	feature schema/arithmetic, label geometry rebuilt from the raw `.mat`, leakage, splits, Platt + bands, artefact schema and size budget, app-asset byte-equality, provenance, and the model-selection rule re-derived from `tuning.json`	`ml/future_risk/tests/test_future_risk.py` (93 tests — see §2)

---

5. NOT EXECUTED — declared honestly (no device available)

There is **no emulator or AVD** in this environment. The following were **compile-checked only** (`:app:compileDebugAndroidTestKotlin` = BUILD SUCCESSFUL) and have **never been run**:

ID	Check	Status	Reason
INST-01	`DatabaseMigrationTest` — opens a real v1 DB, runs `MIGRATION_1_2`, asserts a saved zone survives and `outbox_events` is created	NOT EXECUTED (compiles)	needs a device/emulator
INST-02	`AppDatabaseInstrumentedTest` — on-device persistence smoke test	NOT EXECUTED (compiles)	needs a device/emulator
INST-03	`StatusBannerInstrumentedTest` — Compose UI banner test	NOT EXECUTED (compiles)	needs a device/emulator
INST-04	OpenStreetMap tiles actually render on screen	NOT EXECUTED	needs a device + network; the projection maths is unit-tested but tile download is not
INST-05	Phone vibration / haptic patterns	NOT EXECUTED	emulators have no vibrator; needs a physical phone
INST-06	Google Maps basemap renders with a real API key	NOT EXECUTED	needs a device + a Google Maps API key (none is configured); the branch is compile-checked and the zone/trail overlays are the same code the key-less map uses
INST-07	Create Geo-Fence + Tracking maps open **on the user's own area** (live fix → draft vertex → zone → last-known location)	NOT EXECUTED	needs a device with location; the centring rules are plain Kotlin but the on-screen result is not observable here
INST-08	The "Add 3 sample areas near me" button creates the zones on tap and they appear on the map	NOT EXECUTED	needs a device; the geometry it persists **is** unit-tested (`SampleAreasTest`, MAP-01…MAP-06) but the button wiring is not
INST-09	The on-disk tile cache actually serves a map after the app is restarted with no network	NOT EXECUTED	needs a device + a first online session; the cache's file behaviour **is** unit-tested (`TileDiskCacheTest`, MAP-11…MAP-16) but no real tile round-trip was observed
INST-10	The "Check map servers" diagnostic and the **provider** fail-over recover a blocked host on a real network	NOT EXECUTED	needs a device; the provider order/URLs/keys **are** unit-tested (`TilePolicyTest`) and all **four** providers answered `HTTP 200` from the build machine, but device behaviour is not observed
INST-11	Zoom in/out buttons change the map zoom	NOT EXECUTED	needs a device; trivial UI wiring, no logic of its own
INST-12	The **blue** "you are here" marker is actually visible over the tiles, and the hollow grey ring appears when there is no live fix	NOT EXECUTED	needs a device. The colour is a theme constant and the marker drawing is compile-checked, but "can a person see it over real tiles" is exactly what cannot be judged from this machine — it is the reported bug
INST-13	The crash-report dialog appears after an unexpected close and its Copy button yields the trace	NOT EXECUTED	needs a device plus a real crash; the file round-trip is plain file I/O but the dialog and `Application` wiring are not exercised here
INST-14	Sustained panning/pinching does not get the app killed (tile-memory bounding + request de-bouncing)	NOT EXECUTED	needs a device; `providersToTry` **is** unit-tested (MAP-21…MAP-22) but memory behaviour under a real gesture is not observable here

ID	Check	Status	Reason
INST-15	The watch screen's SOS-delivery note renders the **real** persisted session state (address set or not, signed in or not, last flush outcome)	NOT EXECUTED	needs a device; a plain text block driven by the persisted session, compile- and JVM-checked only
INST-16	The `CameraUpdateFactory is not initialized` crash is actually gone — the reported "blue marker appears, then the app closes"	NOT EXECUTED	needs a device. The guard plus `runCatching` around `followLatestFix` / `moveTo` / `fitPoints` is compile- and JVM-verified, but only a device can confirm the *reported* crash no longer occurs
INST-17	Live sensor-driven movement/fall inference runs on hardware (accelerometer + gyroscope → 24 features → shipped forest → gates)	NOT EXECUTED	needs a physical device with both sensors. The pipeline is JVM-tested against recorded windows (MOV-01…MOV-18) but no sensor has ever been read by this project
INST-18	The movement/fall card and the approach-briefing card render on the Tracking screen (activity + confidence, peak g, impact score, and every risk factor with its points)	NOT EXECUTED	needs a device; the state maths behind them is unit-tested (MOV-04…MOV-18, RISK-01…RISK-12) and the Compose code compiles, but the rendered cards were never seen
INST-19	The default key-free basemap and the distance-to-boundary pill appear on a real screen	NOT EXECUTED	needs a device + network. OpenStreetMap answered `HTTP 200` from the build machine and the pill is driven by the unit-tested briefing, but the rendered result is not observable here
INST-20	The **backend connection (advanced)** screen on a device: entering a base URL, **Test connection** actually reaching `/api/v1/health` and reporting Connected/Offline, and seeing the real profile / delivery totals / latest SOS render (rather than demo text)	NOT EXECUTED	needs a device **and** a reachable server. The client behind it is JVM- and real-socket-tested (AUTH-01…AUTH-14, BE-01…BE-07) and the live test below drives it here, but the Compose screen was never rendered on a device
INST-21	The exact network path a device uses — emulator loopback `10.0.2.2` or a LAN IP over Wi-Fi — delivers an SOS to the authority	NOT EXECUTED	needs a device. `AuthorityLiveBackendTest` proves the *client* against a real server over a real socket on the JVM (see §7.1); the device's own `HttpURLConnection` and its loopback/Wi-Fi routing are not exercised here
INST-22	The **future-risk card** renders on the Tracking screen (horizon, band, calibrated probability, contributing factors with the non-causal caveat, model version, timestamp, and the "Insufficient data"/"Prediction unavailable" states), and the **authority dashboard's predictions panel** renders the rows it receives	NOT EXECUTED	needs a device for the card and a browser for the dashboard. The card's wording is pinned instead by `FutureRiskCardModelTest` (18 tests) and the inference by `FutureRiskReferenceTest` (6), both on the JVM; the dashboard's `renderRisk` path is syntax-checked with `node -e` but was never opened in a browser
INST-23	Live sensor → future-risk inference on hardware (accelerometer → 50-sample window → 22 features → shipped forest → calibrated band)	NOT EXECUTED	needs a physical device. `FutureRiskMovementIntegrationTest` covers the wiring on the JVM against recorded windows, and `FutureRiskReferenceTest` proves the shipped asset agrees with scikit-learn, but no sensor has ever been read by this project
INST-24	The **tourist sign-in + create-account** screen on a device: the sign-in form with its show/hide PIN control, the create-account form (name / phone / PIN / PIN confirmation / add-and-remove optional emergency contacts), the per-field error text, the "*still needed*" line under a submit button, the registration-success card with its **copy** button and sign-in instructions, and the dashboard's **Registered Tourists** panel rendering the rows the API returns	NOT EXECUTED	needs a device **and** a browser reachable to the dashboard. The logic behind the form is JVM-tested (`RegistrationFormTest` 18, `AuthorityViewModelTest` 11) and the production client is driven against a real server on the JVM (the live case in §7.1, which registered `TR-2026-00001` and read the dashboard row back), but no Compose screen was rendered and the panel's `renderTourists` path is only syntax-checked with `node --check`
INST-25	The **role separation as a person experiences it**: the Welcome screen's two choices, a returning tourist being offered *Continue* and landing on the safety home, **Sign out** returning to the Welcome screen with no *Continue*, officer sign-in landing on the console, and the console being unreachable from the tourist path	NOT EXECUTED	needs a device. The state machine behind it is JVM-tested (`OfficerViewModelTest` 10, `AuthoritySessionStoreTest` 8) and the server-side boundary is proven live (a tourist token gets **403** from the officer route, §7.1), but no screen was rendered and the navigation gate was never driven on a device

The correct way to run these later:

```
# with a phone connected (USB debugging on) or an emulator started
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:connectedDebugAndroidTest"
```

---

6. ML evaluation results (measurements, not unit tests)

These are produced by the ML scripts on the **real** datasets. They are reported here for completeness; they are **not** pass/fail tests.

6.1 Trajectory anomaly — GeoLife (unsupervised)

Produced by `ml/evaluate.py` on the **real** GeoLife data (held-out, trajectory-disjoint test split).

Quantity	Value	Source
Held-out test points / trajectories	369,247 / 300 (0 group overlap with train)	`ml/outputs/eval/eval_report.json`
Injected teleport points	17,462	same
ROC-AUC (teleport injection)	**0.8922**	same
Average precision	**0.6763**	same
Precision@k / Recall@k (k = 17,462)	**0.6165** / **0.6165**	same
Label-shuffle control ROC-AUC	**0.5007** (≈ chance, sanity check)	same
Baseline flagging (contamination 0.02)	threshold 0.5299 → 2.00 % flagged	same

> **What these mean and do not mean.** A high injection ROC-AUC means the detector separates *injected GPS teleports* from normal movement. It does **not** mean the model detects real danger. GeoLife has **no anomaly/danger labels**, so accuracy/F1 against real anomalies **cannot be computed** and is deliberately **not** reported. The model output is a *movement anomaly score*, never a danger probability. See `docs/model_card.md`.

6.2 Movement recognition and impact detection — UCI HAR (supervised)

Produced by `ml/har_train.py` and `ml/har_fall.py`. Unlike §6.1 these **do** have real labels, so accuracy, macro-F1 and confusion matrices are computable and are reported. Full method, per-class tables, embedded confusion matrices and limitations: **`docs/model_card_movement.md`**.

Activity recognition, 24 time-domain features, evaluated on **2947 held-out windows from 9 subjects never seen in training** (subject-disjoint, verified: `subject_overlap: []`):

Model	Features	Trees	Accuracy	Macro-F1	Source
On-device (shipped)	**24**	**32**	**0.8351**	**0.8333**	`ml/outputs/har/train/har_train_report.json`
561-feature reference	561	300	0.9287	0.9271	same
Single decision tree	24	1	0.7967	0.7943	same

Per-class, shipped model: WALKING F1 0.7963 · WALKING_UPSTAIRS 0.7277 · WALKING_DOWNSTAIRS 0.8575 · SITTING 0.8020 · STANDING 0.8162 · **LAYING 1.0000**.

Impact / fall-signature detector, 40 trees, 1848 held-out windows (924 positive):

Quantity	Value	Source
ROC-AUC	**0.9999**	`ml/outputs/har/fall/har_fall_report.json`
Average precision	**0.9999**	same
F1 @ 0.5	**0.9962**	same
Precision / recall @ 0.5	0.9957 / 0.9968	same
Shuffled-label control ROC-AUC	**0.5198**	same
Export fidelity (`har_verify_export.py`)	max probability error **4.97e-07** (activity), **4.74e-07** (fall); **0** argmax mismatches	console output, reproduced in this run

> **What these mean and do not mean — the same warning as §6.1, and it matters more here. UCI HAR contains no falls.** The detector's positives are **impacts synthesised into real recorded windows**, so these numbers measure detection of *that injected signature* on real held-out subjects — they are **not** a clinical fall-detection result and must not be presented as one. The shuffled-label control at 0.5198 is what shows the near-perfect score is the model finding the signature rather than exploiting a leak. The named upgrade path is SisFall / UniMiB SHAR / MobiFall, which contain genuine falls. The activity model *does* use real human labels, so its numbers are ordinary supervised accuracy.
>
> **The approach-risk score shown when nearing a restricted area is deliberately *not* here**, because it is not a model: it is a published weighted checklist (`ApproachRiskModel`) with unit-tested weights, and no labelled outcome data exists against which it could be fitted or scored.

6.3 Future risk prediction — UniMiB SHAR (supervised, real recorded falls)

Produced by `ml/future_risk/train.py` (hyper-parameters selected by `ml/future_risk/tune.py`). This is the only model here that forecasts an event **before** it happens: *given the last second of phone accelerometer data, will a hard fall impact land within the next second?* Full method, calibration tables, per-activity false alarms and the complete limitations list: `docs/future_risk_prediction.md`.

Dataset: **UniMiB SHAR** — 30 subjects, 9 activities of daily living + **8 real fall types**, 50 Hz, **real recorded falls** (not injected). 1,980 trials read, **1,979 used**; one recording was corrupt and is counted (see below). Subject-disjoint 60/20/20 split, seed 42, **18/6/6 subjects, disjoint: true**.

Quantity	Value	Source
Windows (train / validation / test)	31,122 / 9,591 / **10,415**	`ml/outputs/future_risk/metrics.json`
Test subjects / positives	6 (never trained on) / 1,339	same
Positive rate (all windows)	0.126310 (6,458 / 51,128)	same
Majority-class baseline PR-AUC	0.128565 (= the positive rate)	same
Logistic-regression PR-AUC	0.659633	same
Hist gradient boosting PR-AUC / ROC-AUC	**0.815000 / 0.945913**	same — **not shipped** (cannot run on-device)
Shipped random forest (calibrated) — precision / recall / F1	**0.6707 / 0.7498 / 0.7080**	same
Shipped random forest — PR-AUC / ROC-AUC	**0.764569 / 0.936848**	same
Confusion @ HIGH band (tp / fp / fn / tn)	1,004 / 493 / 335 / 8,583	same
Trial-level sensitivity	**0.929577** (264 of **284** held-out fall trials flagged)	same
ADL false alarms / minute (0.2 s stride)	**16.295725**	same
ADL false alarms / minute (app cadence 0.64 s)	**5.092414**	same
Post-impact diagnostic alarm rate	0.074227 (970 windows)	same
Brier score (raw → calibrated)	0.062600 → **0.056675**	same
Risk bands (validation-fitted)	HIGH ≥ **0.314046**, MODERATE ≥ **0.069201**	same
Shuffled-label control ROC-AUC / PR-AUC	**0.477430 / 0.129885** (≈ chance)	same
Shipped asset size (budget 2,000,000 bytes)	**1,974,297 bytes** / 120 trees / 66,894 nodes	same
Model-selection cost of the size budget	**−0.031** validation PR-AUC (0.875909 → 0.845112)	`ml/outputs/future_risk/tuning.json`

False alarms by activity, so the failure mode is visible rather than averaged away (held-out ADL windows):

Walking	Running	Upstairs	Downstairs	Jumping	Sitting down	Stand up (sofa)	Lying down	Stand up (floor)
0.011076	0.016412	0.025335	0.044248	**0.160615**	**0.156385**	0.027586	**0.127368**	0.031762

Data quality, reported not hidden: the corrupt recording (subject 10 / `Jumping` / trial 1, **9,369 of 9,624** values non-finite) was **dropped and counted**, and **6** fall trials that never reach the 2.0 g impact threshold are excluded from the positives (`n_fall_trials_without_impact`).

> **What these mean and do not mean.** The labels are **falls in a laboratory**, not tourist incidents, and the positives are real fall impacts so no injection caveat applies — but this is still **not a deployment-ready fall alarm**: **5.09 false alarms per minute** of ordinary activity, over-confident mid-range probabilities, 30 healthy adults carrying a phone, a **one-second** horizon only, and an untested "dropped phone vs falling person" failure mode. A HIGH band is an **informational UI state** and a queued summary; it raises no alarm and **never** triggers an SOS (`backend/tests/test_risk.py` asserts that a HIGH prediction leaves `latestSos` empty and `state != "SOS"`). The shuffled-label control at 0.477 is what shows the pipeline leaks nothing.

---

7. Reproduce everything

```
cd D:\TouristSafetyMonitoring-src

# Android JVM suite: 548 tests, 0 failures, 0 errors, 7 skipped (50 classes)
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:testDebugUnitTest"

# ...and with the opt-in live tests pointed at a running server (they all run and pass):
```

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:testDebugUnitTest" `
    -Dtsm.e2e.baseUrl=http://127.0.0.1:8080 -Dtsm.e2e.credentials=backend\var\DEV_CREDENTIALS.txt

# Instrumented sources compile-check only (nothing runs: no device or emulator here)
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:compileDebugAndroidTestKotlin"

# Debug APK -> app\build\outputs\apk\debug\app-debug.apk  (20.33 MB), authority address baked in
# for this machine's Wi-Fi on first launch:
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 ":app:assembleDebug" `
    -PAUTHORITY_URL=http://192.168.1.102:8080

# Authority backend: in-process rule suite, then the real-subprocess acceptance harness
python -m pytest backend\tests -q          # 265 pass
python backend\verify_integration.py       # 41/41 PASS (starts its own server on a free port)

cd ml
python -m pytest tests -q          # 86 pass (GeoLife/HAR pipeline + OSM, weather, history, transport)
python -m pytest ../ml/tests ../ml/future_risk/tests -q   # 179 pass (from the repo root)
python har_verify_export.py        # exported JSON vs scikit-learn: ALL PASS
python osm_eda.py                  # fetch real OSM (needs network; uses the cache otherwise)
python weather_eda.py              # real weather (Open-Meteo; OpenWeatherMap if a key is set)
python history.py                  # historical trajectory + zone-crossing trends
python transport_mode.py           # supervised transport-mode model (not a risk model)
python future_risk/train.py        # supervised FUTURE fall-impact model -> outputs + app assets
python future_risk/tune.py         # the 65-config validation search under the size budget
powershell -File .\run_all.ps1     # full offline ML pipeline (-Network for the two fetches, -FutureRisk for this model)
```

APK built by this run: **20.33 MB** (21,317,914 bytes), SHA-256 `F0786379EC1917F9004E03F72E9317FB0C0F97B4B7D4BF9F312A911D3BE5D239`, copied to the project root, `Desktop` and `Downloads` as `app-debug.apk`. It carries `BuildConfig.DEFAULT_AUTHORITY_URL = "http://192.168.1.102:8080"` (the build machine's Wi-Fi address), verified present in the packaged `classes8.dex` alongside the constant name, and it declares `android:networkSecurityConfig` → `res/xml/network_security_config.xml` (`cleartextTrafficPermitted="true"`), confirmed by dumping the shipped manifest with `aapt2`.

7.1 The live backend integration, actually exercised this round

Three separate layers were run, in increasing order of realism:

1. **Backend rules, in-process** — `python -m pytest backend\tests -q` → **265 passed** (~110 s). A Flask test client, a temporary SQLite file and a frozen clock; no network.
2. **Backend acceptance, real subprocess on a real socket** — `python backend\verify_integration.py` → **41/41 PASS**. It starts its own server on a free port, signs in as both roles, **self-registers a tourist from a phone that has no credential and signs in with the id and PIN the service minted**, delivers an outbox batch (a simulated SOS, a zone alert, a position report and an **on-device future-risk summary**), checks idempotency and SOS acknowledgement, reads the prediction back from `/authority/risk` and the tourist detail and confirms it raised **no** SOS, confirms a duplicate phone is 409 and a `role: "authority"` registration is 400, **restarts the server to prove persistence** (including the self-registered account), and fetches the dashboard.
3. **The production Kotlin client against a real server** — `AuthorityLiveBackendTest`, run with `-Dtsm.e2e.baseUrl=http://127.0.0.1:8080` and `-Dtsm.e2e.credentials=backend\var\DEV_CREDENTIALS.txt` against the `python backend\run.py` started for this pass. Result: **7/7 passed,** `skipped="0"` (the same seven that skip in a normal run):

- "*a self-registered tourist appears on the authority dashboard and then reports a position*" — the production client registered a brand-new tourist over a real socket (unauthenticated — the request carried **no** bearer token), received `TR-2026-00001`, signed in with that id and the PIN it chose, and the officer's authenticated `GET /authority/tourists` showed it exactly once with `registrationSource="SELF"`, `registrationStatus="REGISTERED"`, `hasPosition=false`, `latestPosition=null` and `trackingStatus="AWAITING_FIRST_REPORT"` — *registration and tracking are separate facts*. After the phone delivered a real-GPS `POSITION` report the same row became `trackingStatus="OK"`, `hasPosition=true`, `state="OK"` with `latestPosition: {18.5204,73.8567,simulated:false}`.
- "*an SOS raised on the phone reaches the authority, persists, and is never duplicated*" — the client signed in as `T-1001` (43-char token), flushed the outbox (`SyncReport(inspected=1, delivered=1, retried=0, rejected=0)`), received a server event id, and re-delivering the same batch was reported `DELIVERED` with **one** row in the officer queue for our event. `/me` returned `latestSos=OPEN simulated=true`. (The absolute counts depend on which tests ran first, so they are read back and compared rather than asserted as fixed numbers.)
- "*a routine position report reaches the authority's map*" — a real-GPS `POSITION` event was queued and delivered, and `GET /me` reported `latestPosition: 18.5204,73.8567 simulated=false`.
- "*a tourist can set a PIN and then sign in with it instead of the password*" — password sign-in, `PUT /me/pin`, then a PIN sign-in that returned `subjectId=T-1001`; a wrong PIN returned HTTP 401.
- "*a dead backend leaves the SOS queued and reports why*" — pointed at a closed port, the event stayed queued with reason "*The authority at 127.0.0.1:1 is unreachable: Connection refused: getsockopt*".

- *"an on-device future-risk summary reaches the authority and is never an incident"* — a `RISK` summary built by the production `OutboxPayloads.futureRisk(…)` was queued and delivered (`SyncReport(inspected=1, delivered=1, retried=0, rejected=0)`); `GET /api/v1/authority/risk` returned exactly **one** row for that prediction instant, labelled `kind="PREDICTION"` with `modelVersion="future-risk-unimib-v1"` , `horizonSeconds=1.0` , `probability=0.62` and both contributing factors; the overview counted `riskAssessments=1` and `riskElevated=1` **while** `sosTotal` **stayed unchanged** — the run logged `[e2e] overview counts after a HIGH forecast: {…,"riskAssessments":1,"riskElevated":1,"sosTotal":1,…}` . A forecast is not an incident.
- *"the officer console reads the register and a tourist token is refused it"* (new this round) — the production client signed in as an **officer**, read `GET /api/v1/authority/tourists` through the very method the in-app console calls (**every** returned row asserted to carry an id, a registration status and a tracking status, and to have a position only when it says it has one), and then presented a **tourist's** token to the same route and received **HTTP 403**. Role separation is a server fact, not a hidden screen.

These are recorded in full in `app/build/test-results/testDebugUnitTest/TEST-…AuthorityLiveBackendTest.xml` (`skipped="0"` when run with a base URL).

> **What is still NOT executed on this layer:** the same flow from an **Android device** (INST-20, INST-21, INST-25). The Kotlin client, its HTTP stack on the JVM, and the server were both real; the device's own `HttpURLConnection` , its loopback `10.0.2.2` routing, and the Compose screens were not. In particular the new **Welcome / role chooser**, the **tourist sign-in + create-account** form with its success card and copy button, and the **in-app officer console** were compile-checked, unit-tested at the ViewModel and client level, and exercised through the production client on the JVM, but never rendered on a screen — no emulator or device is available here.